Report Exerise 02

Task 1

Task 1.1

I implemented the counter in c++ with 2 message queues, so every process has a send and receive queue. At first a message gets send and then the process waits for the other one to send a message to have a somewhat synchronous start.

Then there is a loop to count, where at first each process increments its private counter, waits a bit, then send the private counter and drain everything from the incoming message queue. While draining it checks if it was there was a final message and else just does the `private_counter=max(private_counter, received_counter)`. After we incremented the counter as often as desired we send a done message and drain the queue till we receive a done message. After that the counter gets updated a last time to make synchronize it.

Code for Task 1.1 in `src/replicatedCounter.cpp`.

Task 1.2

timestep	counter A	counter B
0	0	0
1	1	1
2	1	1

The entire point is, that after we stop sending both local counters are the same, which is exactly what max is good for. I assume the correct value should be 2, so the fact that they send the same value does not mean that both are correct and the resulting value is correct

Task 1.3

- A sync message is lost The synchronisation beeing lost is not that big of a deal except is was the one that should start the sending or the one that should mark the final message, since then one program will deadlock. For the other syncs, it will just mean the counter will be out of sync for 1 step and again in the next step.
- The same message is received twice

Nothing will happen, the mqueue is always drained, therefore this will do nothing

- An older message arrives after a newer one

Again no problem since the queue is drained every step

Task 1.4

The fix is that every process has a local counter A and B and sends the other process this information, and only updates one of the variables. The merge is still with max on both A and B and only in the end it gets summed up for the total count.

Task 2

Task 2.1

For this I just start a single process, that creates a file. It then forks, and both processes now try to lock the file, if they aquire the lock read the value, increase it and write it back. I have done this with the posix `lockf` function.

Task 2.2

I mean this is just a classic race condition. Most often it will result in one increase beeing overwritten, but it could also produce some garbage value.

I used files in combination with the posix lockf, which lets one process aquire the lock, while the other yields until the file gets unlocked.

Task 3

Task 3.1

Setup: both processes run on the same machine (Linux, 8 cores), each does 100 increments, there are no artificial waits anymore. Every implementation was run 20 times. The tables show the median time per process in µs, with min–max in brackets.

Replicated counter (POSIX message queues):

	1.1 (max)	1.4 (one slot per node)
Local updates (100 increments)	2.0 (1.6–2.3)	1.6 (1.5–1.7)
Sync during the loop (100 sends + draining)	114.6 (58.2–134.6)	91.5 (80.6–105.8)
Final sync (until both replicas agree)	1.2 (0.3–349.9)	0.9 (0.3–350.0)
Total	140.4 (97.8–446.8)	99.6 (89.6–449.7)

Single-copy counter (shared file protected with `lockf`):

	1 process	2 processes concurrently
Waiting for the lock (100 `lockf` calls)	29.7 (28.3–33.5)	138.1 (47.8–218.7)
Update (read, write, unlock)	97.4 (95.0–114.8)	129.2 (104.0–160.7)
Total	131.2 (129.3–154.5)	269.1 (172.9–367.6)

In the replicated approach the local update is just an increment in memory (about 0.02 µs, basically the resolution of the clock), so almost all of the time goes into synchronization: every increment needs a `mq_send` and at least one `mq_receive`, which are system calls of about 1 µs each. The final sync is usually around 1 µs because both nodes run almost in lockstep. In the slow runs (up to 350 µs) one node was scheduled later and the other one had to wait for its done message. In some of those runs a node also ran ahead and filled the queue (10 messages), so updates were dropped (up to 71). The final values were still correct, since each message carries the full state.

In the single-copy approach every increment needs lock, read, write and unlock, so about 1.3 µs per increment even without contention. With two processes the time spent waiting for the lock grows by a factor of about 4.6 and the total time per process doubles, because a process often has to wait while the other one holds the lock. So concurrent updates directly slow down every update, while in the replicated approach a node never waits for the other one to increment.

All measurements are on a single machine. Over a network every message or access to the store costs a round trip (0.1–1 ms instead of 1 µs). The replicated node only sends its state and continues, while the single-copy node has to wait for the store on every increment, so the difference would become much larger.

Task 3.2

1. In the replicated counter the synchronisation happens when a node receives a message so every time a max is performed. Additionally it syncs at the beginning and the end to ensure it starts and ends at about the same time. For the single-copy counter synchronisation happens at every counter increase step, with the lockf
2. This is not a big deal, it will just continue normally and sync up once it is working again.
3. The nodes will be stale until it becomes available again
4. The single-copy would be better here since, reads don't require mutual exclusion so the resource theoretically an be read anytime, but the writing is the critical part that needs to be synced. For replicated state the node updates its own copy and only once the sync message gets delivered the other node can see the change, in contrast in the single copy.